

35

LangChain

Building multi-step apps with LLMs

Chain calls, add memory and tools — the road to agents

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A framework for multi-step LLM apps
Core idea	Chain steps together into workflows
Adds	Memory, tools, agents on top of LLMs
Best for	Apps with logic, not just one LLM call
Leads to	AI Agents (your next project area)

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is LangChain?	Beyond a single LLM call
2	The Idea of a Chain	Linking steps together
3	The Building Blocks	Models, prompts, chains, more
4	Prompt Templates	Reusable, fill-in prompts
5	Memory	Making a chatbot remember
6	Tools	Letting the LLM use the outside world
7	Toward Agents	When the LLM decides what to do
8	A Real Example	RAG with LangChain
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is LangChain?

LangChain is a framework for building applications that use LLMs in more than one step. A single LLM call is simple. But real apps often need a sequence: take input, look something up, ask the LLM, check the result, maybe ask again, then act. LangChain helps you build these multi-step workflows cleanly.

Where LlamaIndex focuses on connecting your data (RAG), LangChain focuses on the wider app: chaining steps, remembering conversations, using tools, and making decisions. It's the framework that leads naturally into AI agents — your next project area.

■ *LangChain is broad — it can do a LOT. Don't try to learn every feature. Focus on the core ideas here (chains, memory, tools, agents). Those carry you most of the way and set up the agents topics next.*

2. The Idea of a Chain

The name says it: LangChain links steps together into a 'chain'. The output of one step flows into the next, like links in a chain or stations on an assembly line. Each step does one job, and together they form a pipeline.

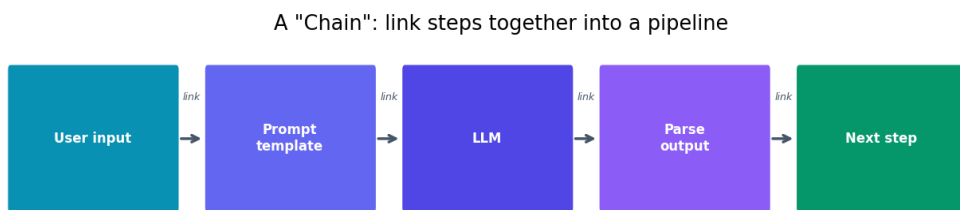


Fig 1 — A chain links steps: input flows through a prompt, the LLM, output parsing, and on to the next step.

A simple chain might be: take a user's topic, fill it into a prompt template, send it to the LLM, then clean up the answer. A more complex chain might do several LLM calls in a row, each building on the last. Chains let you build these flows without messy glue code.

3. The Building Blocks

LangChain gives you a set of building blocks. You combine them to build your app. Here are the main ones.

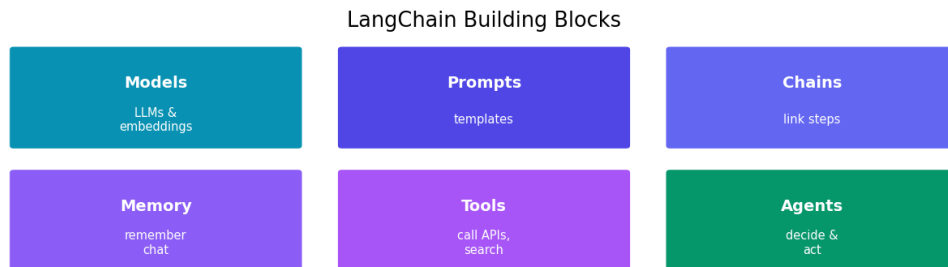


Fig 2 — The main LangChain building blocks, from models up to full agents.

Block	What it does
Models	Connect to any LLM or embedding model
Prompts	Reusable prompt templates with fill-in slots
Chains	Link steps into a pipeline
Memory	Remember earlier messages in a conversation
Tools	Let the LLM use search, APIs, code, calculators
Agents	Let the LLM decide which tools to use and when

These stack on top of each other. Models are the base. Prompts and chains organise how you call them. Memory and tools add abilities. Agents sit at the top, using everything to make decisions. We'll touch each one.

4. Prompt Templates

Instead of writing a full prompt every time, you make a template with blanks to fill in. This keeps prompts consistent and reusable — you write the structure once and just swap the variable parts.

```
from langchain.prompts import PromptTemplate

template = PromptTemplate.from_template(
    'Explain {topic} to a {audience} in 3 sentences.'
)

prompt = template.format(topic='RAG', audience='beginner')
# 'Explain RAG to a beginner in 3 sentences.'
```

Templates are the clean way to handle the prompt engineering you learned in Topic 30. You bake your good prompt structure into a template, then reuse it with different inputs across your whole app.

5. Memory

By default, an LLM forgets everything between calls — each request starts fresh. For a chatbot, that's a problem: you want it to remember what was said earlier in the conversation. LangChain's memory handles this automatically.

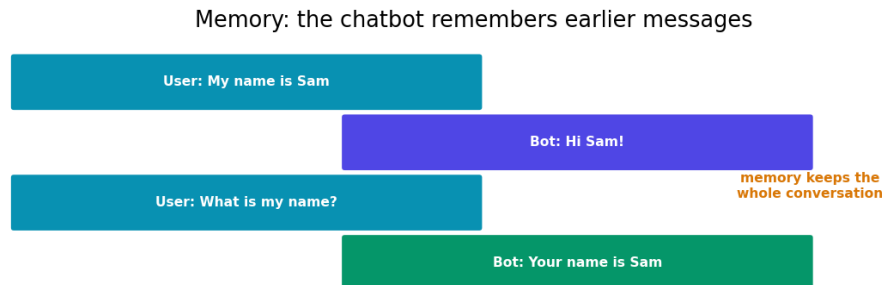


Fig 3 — Memory keeps the conversation, so the bot remembers your name was Sam.

Memory stores the conversation history and feeds it back into each new prompt, so the model always has the context of what came before. This is how a chatbot can answer 'what did I just ask?' or remember your name from earlier. Remember the context window from the LLM topic — memory is how LangChain manages what goes into it across a conversation.

■ *Memory is essential for any real chatbot. Without it, every message is treated as a brand-new conversation with no history — frustrating for users. With it, the bot feels like it's actually following along.*

6. Tools

On their own, LLMs can't do things like search the web, run code, check today's weather, or do exact math. Tools fix this. A tool is a function the LLM can call to reach the outside world. You give the LLM a set of tools, and it can use them when needed.

Tool example	What it lets the LLM do
Web search	Look up current information
Calculator	Do exact math (LLMs are bad at this)
Code runner	Execute code and use the result
Your RAG retriever	Search your documents
Any API	Check weather, send email, query a database

Tools turn an LLM from a closed text-generator into something that can interact with the real world. This is the key idea that makes agents possible — and it's exactly your planned AI Agent project, where you'll wire your RAG chatbot in as a tool the agent can use.

7. Toward Agents

Here's where it gets exciting. So far, YOU decide the steps. An agent flips this: you give the LLM a goal and a set of tools, and the LLM itself decides which tools to use, in what order, to reach the goal. It reasons, acts, sees the result, and decides what to do next.

For example, ask an agent 'what's the weather where the Eiffel Tower is, in Fahrenheit?' It might: use a search tool to find the Eiffel Tower is in Paris, use a weather tool to get Paris's temperature, then use a calculator to convert to Fahrenheit — all on its own. LangChain provides the framework to build these. The next topics cover agents in depth.

■ *This is the bridge to your next project area. An agent = LLM + tools + the ability to decide. LangChain's chains, memory, and tools are the foundation agents are built on. You're now right at the doorstep of agentic AI.*

8. A Real Example — RAG with LangChain

LangChain can build RAG too (like LlamaIndex). Here's a RAG chain, showing how the blocks combine:

```
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA

# 1. Embed + store documents in FAISS
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
store = FAISS.from_texts(chunks, embeddings)

# 2. Build a RAG chain: retriever + LLM
qa = RetrievalQA.from_chain_type(
    llm=my_llm,
    retriever=store.as_retriever(search_kwargs={'k': 3})
)

# 3. Ask questions
print(qa.run('What does the document say about leave?'))
```

■ *See how the pieces you know appear: FAISS for the index, all-MiniLM for embeddings, top-k=3 for retrieval. LangChain just wires them into a reusable chain. Same RAG concepts, framework-assembled.*

9. Common Mistakes

Mistake	Why it's a problem	Fix
Trying to learn it all	It's huge and overwhelming	Focus on core: chains, memory, tools
Using it for one LLM call	Unneeded complexity	Just call the LLM directly
No memory in a chatbot	Bot forgets everything	Add memory
Giving agents too many tools	Confused decisions	Start with a few clear tools
Not knowing what runs underneath	Hard to debug	Understand the steps (you do!)
Ignoring LlamaIndex for pure RAG	More setup than needed	Use LlamaIndex if it's just docs

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
LangChain	framework for multi-step LLM apps
Chain	linked steps forming a pipeline
Prompt template	reusable prompt with fill-in slots
Memory	remembers the conversation
Tool	function the LLM can call (search, math, API)
Agent	LLM that picks tools to reach a goal
Install	<code>pip install langchain</code>
Prompt template	<code>PromptTemplate.from_template('...{x}...')</code>
Vector store	<code>FAISS.from_texts(chunks, embeddings)</code>
RAG chain	<code>RetrievalQA.from_chain_type(llm, retriever)</code>
Run	<code>chain.run('your question')</code>
Best for	workflows, tools, agents

The one idea to remember: LangChain builds multi-step LLM apps by chaining steps together, and adds memory, tools, and agents. It's how you go from a single LLM call to a real application that remembers, uses tools, and makes decisions — the foundation for AI agents.

Next Topic: Agentic AI — the big concept of AI agents that can reason, plan, use tools, and act on their own to complete tasks. This is the cutting edge, and your next project frontier.